Reinforcement Learning for the Continuous Art Gallery Problem

Preliminaries, Policy Architecture, and MDP Formulation

AGP Project — Technical Report

October 26, 2026

Abstract

We train a PPO agent to place point guards in polygonal galleries with holes so that at least 98% of the free space is seen, using as few guards as possible. This report explains every design decision in the current implementation. Section 2 gives the geometry that motivates a two-stage curriculum: coverage is monotone submodular, so a guard's redundancy can only be measured *after* the gallery is saturated, and the dense rewards we use are potential-based, so they telescope into the true objective. Section 3 describes the policy network and why each part of it exists: it keeps the map's spatial layout to the output and factors each placement into a choice of region and a *continuous* point inside it. Section 4 states the MDP and the reward with the exact constants used in code. Section 5 outlines an optional third stage with a relocation action.

Contents

1 Introduction **2**

2 Preliminaries: why two stages **2**
2.1 Galleries, visibility, coverage . . . 2
2.2 Complexity, and what a learned policy can gain . . . 3
2.3 Submodularity: why pruning must come second . . . 3
2.4 Greedy, its guarantees, and the reference count . . . 4
2.5 Why one stage collapses . . . 4
2.6 Potential-based shaping: dense reward without bias . . . 5

3 Architecture **6**
3.1 What the policy has to do . . . 6
3.2 The network . . . 6
3.3 The factored action . . . 8
3.4 The mode head across the stage switch (R5) . . . 10
3.5 PPO configuration . . . 10
3.6 Results . . . 11

4 MDP formulation and reward **11**
4.1 Instances . . . 11
4.2 State, actions, termination . . . 11
4.3 Observation . . . 12
4.4 Reward . . . 13
4.5 Schedule . . . 14
4.6 Weight calibration . . . 14

5 Optional stage 3: relocation 14

1 Introduction

The agent sees an image of a gallery and the guards placed so far. At each step it either adds a guard at a point, deletes the guard nearest to a point, or stops. Training runs in two stages on one network (fig. 3):

Saturate (stage 1). Only ADD is allowed and guards are free. The agent learns to cover arbitrary galleries.

Prune (stage 2). DEL and STOP are unmasked, every guard costs reward, and deleting a guard that costs no coverage is paid. The agent learns to remove the guards it does not need.

Three facts shape everything below. The problem is NP-hard, so a learned heuristic is a reasonable target (section 2.2). Coverage is submodular, which fixes the *order* of the two stages (section 2.3). And guards see without a range limit, so whether a point is a good guard position depends on the whole gallery, which shapes the network (section 3). The implementation lives in `agp/`: geometry in `gallery.py`, the MDP in `env.py`, rewards in `rewards.py`, the policy in `policy.py`, and the curriculum in `train.py`. All constants quoted below are the values in that code.

2 Preliminaries: why two stages

2.1 Galleries, visibility, coverage

Definition 1 (Gallery and free space)**.** A gallery $G = (P, \mathcal{H})$ is a simple polygon $P \subset \mathbb{R}^2$ with a finite set $\mathcal{H}$ of pairwise disjoint polygonal holes inside it. The free space is $\mathcal{F} = P \setminus \bigcup_{H \in \mathcal{H}} \operatorname{int} H$, with area $A = |\mathcal{F}| > 0$.

Definition 2 (Visibility polygon)**.** A guard at $g \in \mathcal{F}$ sees $\mathcal{V}(g) = \{q \in \mathcal{F} : \overline{gq} \subseteq \mathcal{F}\}$.

Visibility has *unlimited range*: a guard sees every point it has an unobstructed segment to, however far, and only the outer wall and the holes block it (fig. 1). `Gallery.visibility_polygon` computes $\mathcal{V}(g)$ exactly. It casts one ray towards every vertex of P and of every hole, plus one ray $\pm 10^{-6}$ rad to either side, so that both edges meeting at an occluding (reflex) corner are captured. Each ray is $3\times$ the bounding-box diagonal long, so effectively infinite, and is cut at its first boundary hit. The hits, taken in angular order, form $\mathcal{V}(g)$, which is clipped back to $\mathcal{F}$.

Definition 3 (Coverage)**.** For a finite guard set $S \subset \mathcal{F}$ let $\mathcal{C}(S) = \bigcup_{g \in S} \mathcal{V}(g)$ and $f(S) = |\mathcal{C}(S)|/A \in [0, 1]$. A gallery is *solved* when $f(S) \geq c^\dagger$ with $c^\dagger = 0.98$.

The threshold $c^\dagger < 1$ is deliberate. Full coverage can require a guard for a sliver of a few square units behind a hole, and that last guard dominates the count without telling us anything about placement quality. At 98% the greedy baseline solves 598/600 training and 88/90 test galleries under the 16-guard budget; the rest stop between 97.7% and 98% because no candidate point adds new area (section 2.4).

Two representations of the same geometry. Every quantity that enters the *reward* is computed exactly with Shapely polygons: $f(S)$, the area a new guard adds, and the area lost when a guard is removed. The *observation* is a 64×64 raster of the gallery's bounding box: pixel (r, c) samples the point $(x_{\min} + c\,w/63,\ y_{\min} + r\,h/63)$, where w, h are the box's width and height. The raster is also used for one cheap estimate shown to the agent, the per-guard exclusive area (section 4.3). Exact rewards mean the agent is never paid for raster artefacts; a raster observation means the network can be convolutional.

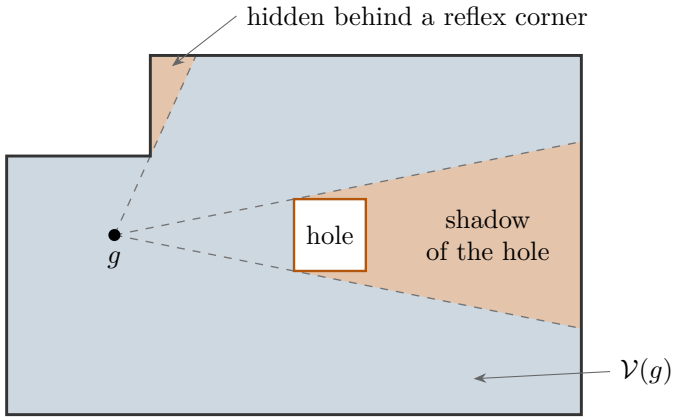


Figure 1: Visibility has unlimited range and is blocked only by walls and holes. The guard g sees the whole blue region, up to the far wall. It cannot see the orange regions: the shadow cast by the hole, bounded by the rays through the hole's two silhouette corners, and a thin wedge behind the reflex corner of the outer wall. Thin wedges like this are what the last few guards of a solution exist to cover.

2.2 Complexity, and what a learned policy can gain

Chvátal's theorem gives $\lfloor n/3 \rfloor$ vertex guards as the tight worst case for a simple n-gon [4, 8]. Finding a *minimum* guard set is NP-hard [10], hard to approximate [6], and with guards allowed anywhere in the polygon the decision problem is $\exists\mathbb{R}$-complete [1]. Exact and near-exact solvers exist but restrict guards to finite candidate sets such as vertices or grid points [5, 15].

This tells us where a learned policy can do better than classical heuristics:

1. **Continuous positions.** Optimal point guards need not lie on any candidate set. A policy that outputs real coordinates can reach positions no candidate-based method can. We keep the action continuous for this reason (section 3.3).
2. **Pruning.** Greedy covers are not minimal, so removing redundant guards recovers slack that greedy leaves behind (section 2.4).

2.3 Submodularity: why pruning must come second

Proposition 4 (Monotone submodularity)**.** *$f(\emptyset) = 0$, f is monotone, and for all $S \subseteq S'$ and every g,*

$$f(S \cup \{g\}) - f(S) \;\geq\; f(S' \cup \{g\}) - f(S').$$

Proof. With $\nu = |\cdot|/A$, the marginal gain is $\nu(\mathcal{V}(g) \setminus \mathcal{C}(S))$. Since $\mathcal{C}(S) \subseteq \mathcal{C}(S')$, the set being measured can only shrink as S grows. □

Two quantities drive the design. For a candidate $g \notin S$ and a placed guard $g_j \in S$,

$$\delta^+(g \mid S) = f(S \cup \{g\}) - f(S), \qquad \delta^-(g_j \mid S) = f(S) - f(S \setminus \{g_j\}).$$

δ^+ is the *marginal gain* of adding, and δ^- is the *exclusive area* of a placed guard, i.e. what deleting it would cost. A guard with $\delta^- = 0$ is strictly redundant.

Proposition 5 (Redundancy is revealed only after saturation)**.** *Insert guards $g_1, \ldots, g_K$ in order and let $S_j = \{g_1, \ldots, g_j\}$. Then for every j,*

$$\delta^-(g_j \mid S_K) \;\leq\; \delta^+(g_j \mid S_{j-1}),$$

and $\delta^-(g_j \mid S)$ is non-increasing in $S \ni g_j$.

Proof. Apply proposition 4 to $S_{j-1} \subseteq S_K \setminus \{g_j\}$. □

Figure 2 shows both propositions on a real test gallery. The coverage curve flattens as guards are added, and the second guard, which added 28% of the gallery when it was placed, ends up with an exclusive area of zero.

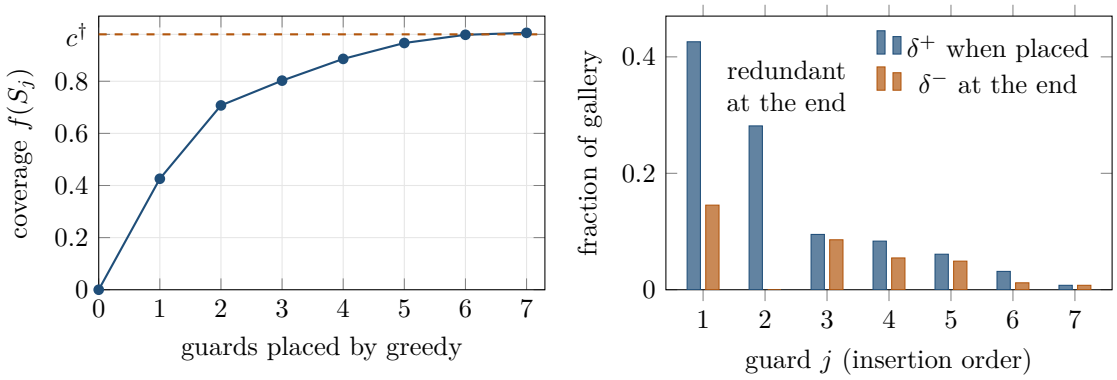


Figure 2: Greedy saturates with diminishing returns and leaves redundant guards behind. Real data: greedy cover of hard test gallery #63. *Left:* each extra guard adds less (proposition 4); the seventh adds 0.8% to cross $c^\dagger$. *Right:* for every guard, the exclusive area at the end (δ^-) is at most its gain when placed (δ^+), as proposition 5 requires. Guard 2 covered 28% when placed and is fully redundant at the end, so the greedy prune removes it ($k_{\text{greedy}} = 7$, $k_{\text{ref}} = 6$). Its redundancy only became visible after guards 3–7 were placed.

Consequence. A guard can only become *cheaper* to delete as more guards are added. A deletion decision made early in an episode sees an exclusive area that is too large; after saturation it sees the smallest value that guard will ever have. Learning to cover first and to prune second is therefore the order in which the pruning signal is most informative, not just a convenience.

2.4 Greedy, its guarantees, and the reference count

The greedy rule "add the guard of largest δ^+" inherits the $(1 - 1/e)$ guarantee for budgeted monotone submodular maximisation [13] and the logarithmic-factor guarantee for full cover, which is essentially the best possible for set cover in polynomial time [7]. Greedy is therefore strong at *covering*. It is weak at *minimality*: by proposition 5, guards chosen early lose exclusive area as later guards overlap them, and greedy never revisits a choice.

Our baseline (`baselines.py`) makes this concrete for each gallery:

1. **Candidates.** One point per vertex of P and of every hole, moved 1% of the diagonal towards an interior reference point, plus 40 uniform interior samples.
2. **Greedy cover.** Repeatedly add the candidate with the largest raster marginal gain until $f \geq c^\dagger$, the budget of 16 is reached, or no candidate adds area. The count is k_{greedy}.
3. **Greedy prune.** Repeatedly delete the guard with the smallest exact δ^- while coverage stays $\geq c^\dagger$. The remaining count is k_{ref}.

k_{ref} is the per-instance reference that the parsimony reward and the reported *guard ratio* k/k_{ref} are measured against. It is an *upper bound* on the optimum, not the optimum: it depends on the candidate set, and the true minimum is NP-hard to compute. A guard ratio below 1 means beating greedy, not proving optimality. On the training set, k_{greedy} ranges over 1–3 (easy), 2–5 (mid) and 3–11 (hard). Mean test-set values are in table 1.

2.5 Why one stage collapses

The objective is to solve the gallery with few guards. With a guard cost w_{par} per guard and a coverage weight w_{cov}, the immediate value of one more ADD is $w_{\text{cov}}\,\delta^+ - w_{\text{par}}$, which is negative

Table 1: Greedy baseline on the 90 test galleries (30 per tier).

tier	mean k_{greedy}	mean k_{ref} (after prune)	mean coverage
easy	1.93	1.93	0.994
mid	2.70	2.67	0.992
hard	6.07	5.97	0.986

whenever

$$\delta^+ \;<\; w_{\text{par}}/w_{\text{cov}} \;=\; 0.15 \quad \text{(our stage-2 weights)}.$$

In fig. 2, guards 3–7 all add less than 15%. Their only justification is the terminal bonus for crossing $c^\dagger$, and a policy that cannot yet cross $c^\dagger$ reliably never collects it. From such a policy's point of view the late guards are pure cost, so it learns to place fewer of them, which makes crossing $c^\dagger$ rarer still. The fixed point of this feedback is "place few guards and fail".

Stage 1 breaks the loop by setting $w_{\text{par}} = 0$: every ADD has non-negative value, the goal bonus is reachable, and the agent learns to cover. Stage 2 turns the cost on only after covering is reliable (fig. 3). This matches the reference locomotion work, where a single-stage dribbling policy reaches 33% success against 86.7% with two stages [16].

2.6 Potential-based shaping: dense reward without bias

PPO needs a dense signal, but dense rewards can change which policy is optimal. We avoid this by making the two dense terms potentials. Let $u(s) = 1 - f(S)$ and $k(s) = |S|$ and define

$$\Phi(s) \;=\; -w_{\text{cov}}\, u(s) \;-\; w_{\text{par}}\, k(s), \qquad F(s, s') = \gamma\Phi(s') - \Phi(s).$$

At $\gamma = 1$, F is exactly the per-step reward we pay: $+w_{\text{cov}}\delta^+ - w_{\text{par}}$ on ADD and $-w_{\text{cov}}\delta^- + w_{\text{par}}$ on DEL. Along any episode from the empty gallery ($u_0 = 1$, $k_0 = 0$),

$$\sum_{t=0}^{T-1} F(s_t, s_{t+1}) \;=\; \Phi(s_T) - \Phi(s_0) \;=\; w_{\text{cov}}\, c_T - w_{\text{par}}\, k_T,$$

which is the objective itself. By the shaping theorem of Ng et al. [14], adding F does not change the optimal policy. We train with $\gamma = 0.995$, for which the sum is $\gamma^T\Phi(s_T) - \Phi(s_0)$; the extra factor γ^T mildly favours short episodes, i.e. fewer guards and a prompt STOP, which is the direction we want anyway.

Design rule 6 (Anneal iff non-potential)**.** A reward term may keep a fixed weight for the whole of training only if it is a potential difference or part of the objective. Any other term biases the optimum and must reach zero before training ends.

Design rule 6 decides the schedule in fig. 3: the coverage and guard-count terms stay on, and the stage-1 scaffolds (frontier, sweep, separation) are annealed to zero.

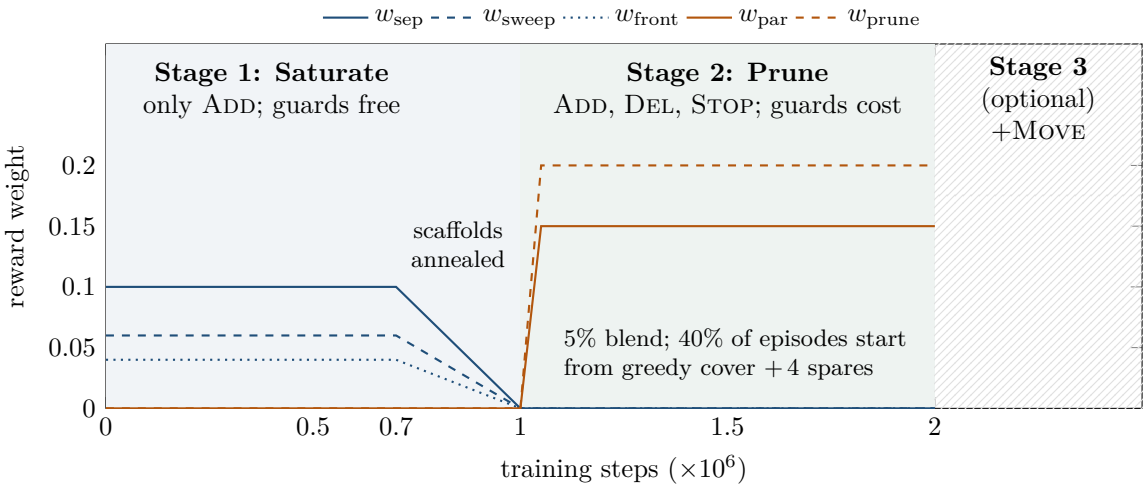


Figure 3: The curriculum and its reward-weight schedule, with the exact values from `rewards.py`. In stage 1 the scaffolds (blue) hold their weight to 70% of the stage and then fall linearly to 0, so the last 30% trains on the unbiased reward (design rule 6). Stage 2 blends from that column to the parsimony weights (orange) over its first 5%, which spares the value function a step change. Not plotted because of scale: the potential term $w_{\text{cov}} = 1$ and the terminal $w_{\text{goal}} = 4$, $w_{\text{fail}} = 2$ are constant; w_{pars} $(0 \to 0.5)$, w_{stop} $(0 \to 2)$ and w_{void} $(0 \to 0.1)$ blend in with w_{par}.

3 Architecture

3.1 What the policy has to do

At every step the policy outputs a mode in $\{\text{Add}, \text{Del}, \text{Stop}\}$ and a point (x, y) in the gallery's bounding box, normalised to $[0, 1]^2$. The geometry of section 2 turns this into five concrete requirements, and each part of the network answers one of them:

R1 Continuous placement. Point guards are continuous (section 2.2), so the action must reach any point, not a fixed candidate set.

R2 Several targets at once. A partly covered hard gallery usually has several separate uncovered pockets. The distribution over points must be able to put mass on all of them, rather than on their average.

R3 Global context. With unlimited range, whether a point is useful depends on what it can see across the whole gallery, not on its neighbourhood.

R4 Fine localisation. The missing 2–4% of a nearly solved gallery is often a thin wedge behind a hole or a reflex corner (fig. 1), a few pixels wide. The network must not blur it away before deciding where to aim.

R5 Unordered modes with masking. Add, Del and Stop are unordered alternatives, and stage 1 must disable two of them cleanly.

3.2 The network

Figure 4 shows the network (`policy.SpatialPolicy`, about 294k parameters). Its organising idea is to *read the placement off a map* rather than compute coordinates from a summary vector: one branch keeps the image's layout down to a 16×16 grid of cells, and a second branch summarises the whole gallery into one vector that is handed to every cell. The mode and the value of the state, which are properties of the whole gallery, are read from the summary.

A 16×16 map, never flattened (R4). The local branch downsamples the 64×64 image only to the cell grid ($G = 16$, each cell 4×4 pixels) and keeps it there. Every cell has its

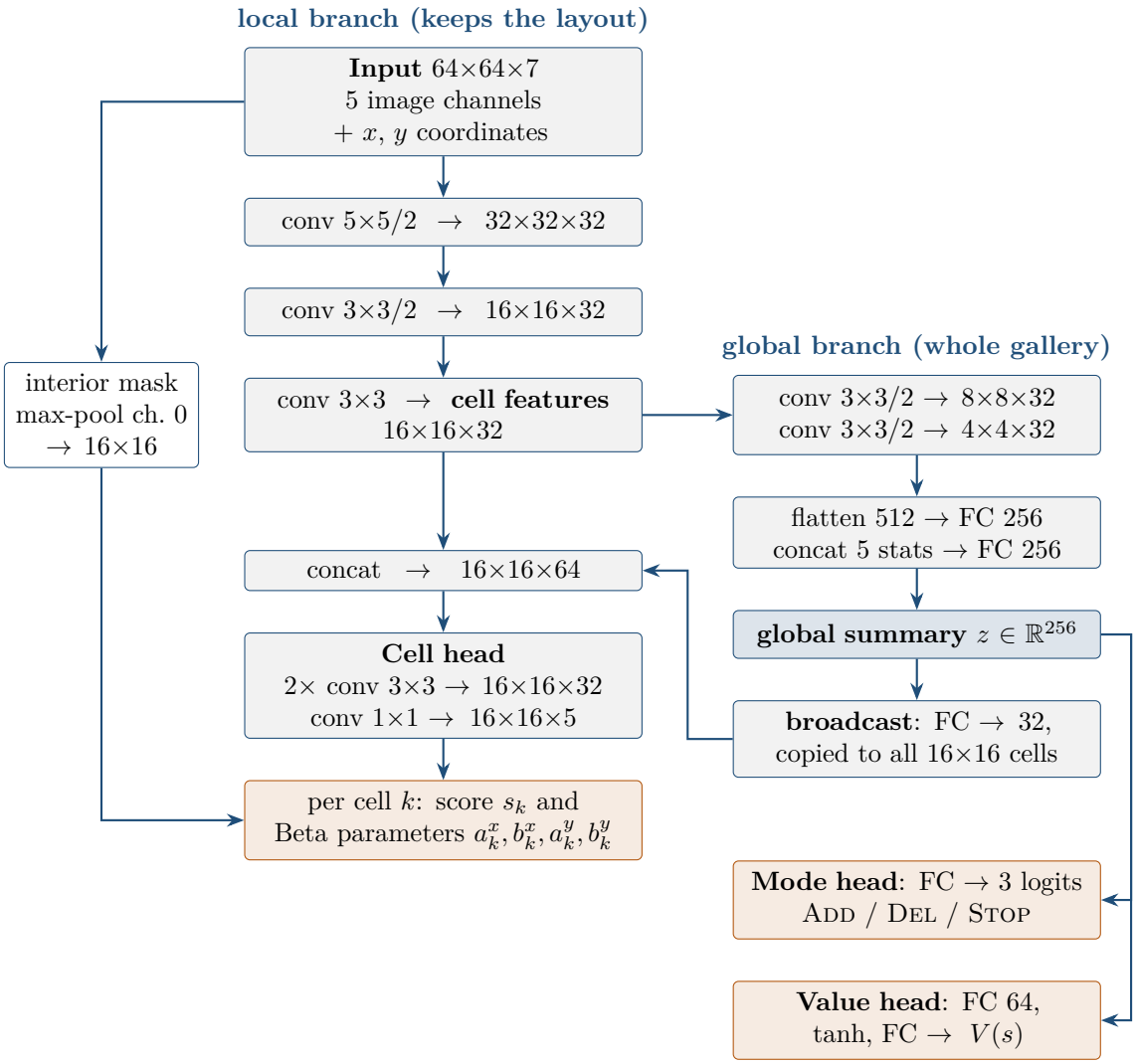


Figure 4: The spatial policy, with tensor sizes. Left: the local branch never flattens the image, so placement is scored per cell on a 16×16 map. Right: the global branch summarises the whole gallery and the scalar stats into z, which is broadcast back to every cell (unlimited-range visibility depends on the whole gallery) and also feeds the mode and value heads. The interior mask removes cells with no free space from the cell distribution. ReLU after every convolution; "/2" is stride 2.

own feature vector, computed from its own neighbourhood, so a thin uncovered wedge stays a feature of the cells it touches and can raise their scores directly. Aiming at a corner is then no harder than aiming at open floor: it is a different cell of the same map. Flattening the image into one vector before choosing a position would force the network to convert "there is a sliver at the top left" into numeric coordinates, which is exactly the kind of fine spatial detail a coarse summary loses.

Coordinate channels. Convolutions are translation-invariant, but visibility is not: a point next to a wall sees differently from the same local pattern in the middle of the room. Two extra input channels hold each pixel's normalised x and y [12], so every cell knows where it is in the gallery.

A global summary broadcast to every cell (R3). Local features alone cannot tell whether a point would see an uncovered region on the far side of the gallery. The global branch continues to 4×4, flattens, and merges the result with the five scalar stats into $z \in \mathbb{R}^{256}$. A linear map of

z to 32 numbers is copied to every cell and concatenated with that cell's local features, so each cell is scored from both its neighbourhood and the state of the whole gallery: what is covered, where the guards are, how much budget is left.

Masking cells outside the gallery. A cell is *live* if any of its 4×4 raster samples lies in the free space (a max-pool of the interior channel). Dead cells, outside P or entirely inside a hole, get logit -10^8 and zero probability (fig. 5). This removes a large and useless part of the search space (in the gallery of fig. 5, 68 of the 256 cells), and the policy never wastes probability on positions that would only be projected back into the gallery and penalised.

Heads and initialisation. The cell head outputs five maps: a score s_k per cell and four Beta parameters, made ≥ 1 by $1 + \mathrm{softplus}(\cdot)$ so that every offset density is unimodal and bounded. The mode and value heads read z, because "stop now" and "how good is this state" are global questions. The last layers of the cell and mode heads are initialised orthogonally with gain 0.01, so training starts from near-uniform cells, near-uniform modes and gently centred offsets.

3.3 The factored action

Index cells by $k = rG + c$ with row r along y and column c along x. The policy samples

$$\mu \sim \mathrm{Cat}(\pi^{\mathrm{mode}}), \qquad k \sim \mathrm{Cat}(\pi^{\mathrm{cell}}), \qquad o_x \sim \mathrm{Beta}(a_k^x, b_k^x), \quad o_y \sim \mathrm{Beta}(a_k^y, b_k^y),$$

$$(x, y) \;=\; \Big(\frac{c + o_x}{G},\; \frac{r + o_y}{G}\Big), \qquad \pi_k^{\mathrm{cell}} \propto \exp(s_k)\, \mathbf{1}[\text{cell } k \text{ is live}]\,,$$

and sends the vector $[\mu, x, y]$ to the environment (fig. 5).

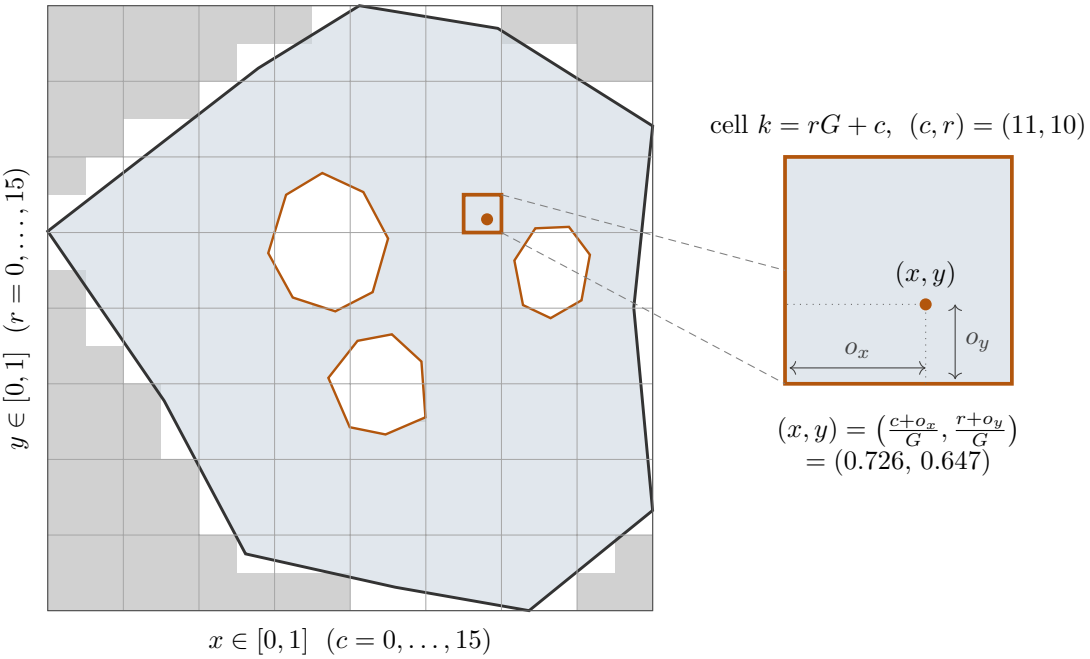


Figure 5: A region is chosen discretely, the point inside it continuously. Left: the 16×16 cell grid over the bounding box of a real gallery (mid test gallery #30). Grey cells contain no free space and are masked. The policy samples a live cell (orange, $(c, r) = (11, 10)$), then a continuous offset from that cell's two Beta distributions, $o_x \sim \mathrm{Beta}(a_k^x, b_k^x)$ and $o_y \sim \mathrm{Beta}(a_k^y, b_k^y)$. Right: the offset fixes the exact point, here $o_x = 0.62$, $o_y = 0.35$. Any point in the gallery can be produced, so the grid organises the choice without restricting where a guard can go.

Why a distribution over regions (R2). When several parts of the gallery are still uncovered, the policy should be able to aim at any of them. A categorical over cells assigns independent probability to every region, so it can put mass on two opposite corners at once. A single Gaussian over (x, y), or any method that outputs one averaged point, cannot: its mean falls between the targets, usually in space that is already covered (fig. 6). The same holds for the deterministic action used in evaluation: the most probable cell is an actual choice of one region, while the mean of a unimodal distribution is a compromise between regions.

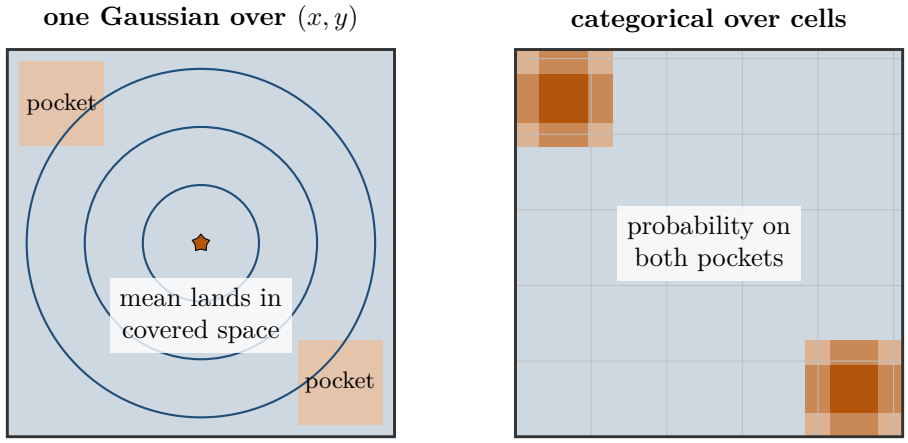


Figure 6: Why the policy chooses a region instead of one averaged point. Two uncovered pockets (orange) sit in opposite corners of an otherwise covered gallery (schematic). *Left:* a single Gaussian that tries to serve both pockets centres between them, so its mean, the deterministic action, lands in already-covered space, and its samples mostly do too. *Right:* the categorical over cells puts high probability (darker cells) on both pockets independently, so every sample aims at real work, and the most probable cell is one of the pockets.

Why a continuous offset inside the cell (R1). The Beta offset makes the action continuous while keeping the density exact.

Proposition 7 (The action is continuous)**.** *For $\mu \neq \textsc{Stop}$, the induced distribution of (x, y) has density*

$$p(x, y) \;=\; G^2 \, \pi^{\text{cell}}_{k(x,y)} \, \text{Beta}(o_x(x)) \, \text{Beta}(o_y(y))$$

on $[0, 1]^2$, where $k(x, y) = \lfloor Gy \rfloor G + \lfloor Gx \rfloor$, $o_x(x) = Gx - \lfloor Gx \rfloor$ and $o_y(y) = Gy - \lfloor Gy \rfloor$. It is positive on every live cell.

Proof. Cells partition $[0, 1)^2$, so each point belongs to exactly one cell and determines its offsets uniquely. Within a cell the map from offsets to (x, y) scales each coordinate by $1/G$, so the Jacobian contributes G^2. Beta densities with parameters ≥ 1 are positive on $(0, 1)$. □

The grid organises *how* the policy decides; it does not restrict *where* a guard can go. This is the difference from candidate-set solvers [5, 15] and from a plain discrete "spatial action map" [17], which would snap every guard to a cell centre and give up advantage (1) of section 2.2. The Beta distribution is the natural choice for a bounded offset: it lives on $(0, 1)$ exactly, so no probability is clipped at the cell edges, which avoids the boundary bias of clipped Gaussians [3].

Log-probability, entropy, deterministic action. Given a stored action $[\mu, x, y]$, the code recovers k, o_x, o_y and computes

$$\log \pi(a \mid s) \;=\; \log \pi^{\text{mode}}_{\mu} \;+\; \mathbf{1}[\mu \neq \textsc{Stop}] \left(\log \pi^{\text{cell}}_{k} + \log \text{Beta}(o_x) + \log \text{Beta}(o_y) \right).$$

The constant $\log G^2$ is dropped because it cancels in PPO's probability ratio. The point term is excluded for STOP because a STOP ignores the point, so including it would only add gradient noise to a decision that does not depend on it. Offsets are clamped to $[10^{-5}, 1-10^{-5}]$, which keeps sampled points strictly inside their cell and the Beta log-densities finite. The entropy bonus is that of the joint choice,

$$\mathcal{H} \;=\; \mathcal{H}(\pi^{\text{mode}}) + \mathcal{H}(\pi^{\text{cell}}) + \textstyle\sum_k \pi_k^{\text{cell}} \big(\mathcal{H}(\text{Beta}_k^x) + \mathcal{H}(\text{Beta}_k^y)\big).$$

The deterministic action used for evaluation is the most probable mode, the most probable cell, and the Beta means $a/(a+b)$ inside it.

3.4 The mode head across the stage switch (R5)

A categorical, not a number. The three modes are separate logits. Moving probability from ADD to STOP does not pass through DEL, as it would if the mode were one continuous output cut into bands: learning "stop once covered" must not require first becoming worse at something else.

Stage 1: masking at the logits. The DEL and STOP logits are replaced by -10^8, so their probability is exactly zero and they contribute nothing to the log-probability or the entropy [9]. The environment also decodes every stage-1 action as ADD, so the MDP is correct for any policy.

Stage 2: priming. Masked logits receive no gradient, so at the switch the mode head is still at its initialisation and would choose among the three modes arbitrarily. Instead, we zero the mode head's weights and set its biases so that $\pi^{\text{mode}} = (0.8, 0.1, 0.1)$ in every state (`-add-prob 0.8`). Stage 2 therefore starts by behaving as stage 1 ended, mostly adding, while sampling DEL and STOP often enough to learn their value. The placement heads are left as stage 1 trained them.

3.5 PPO configuration

Table 2: PPO settings (`train.py`). One model runs both stages; the step counter continues across the switch.

learning rate	3×10^{-4}	discount γ	0.995
environments	16 (subprocesses)	GAE λ	0.95
rollout length	256 per env (4096 per update)	clip range	0.2
minibatch	512	epochs per update	10
entropy coefficient	0.005 (stage 2: 0.01)	value coefficient	0.5
gradient clip	0.5	target KL	off
steps	10^6 per stage	device	Apple MPS

Table 2 lists the settings. $\gamma = 0.995$ is close to 1 because episodes are short (at most 40 steps) and the terminal bonus must reach the first placement. The entropy coefficient is doubled in stage 2 because two of the three modes are new. We do not stop updates early on a KL threshold: a cap of 0.03 ended updates after so few gradient steps that the placement head barely trained. Training runs on Apple's MPS GPU: a forward and backward pass over 512 samples takes 12 ms there versus 600 ms on the CPU. Beta sampling, the one operation MPS lacks, is done on the CPU for the single offset drawn per environment per step. $G = 16$ is a compromise: 256 regions separate distinct pockets while keeping the categorical small enough to explore, and $G = 32$ would double the network's cost without changing what positions are reachable.

3.6 Results

Results pending. The two-stage run with the spatial policy (`runs/agp_spatial`) was in progress when this report was written. Table 3 is a placeholder to be filled from the final held-out evaluation (deterministic actions, 90 test galleries).

Table 3: Held-out evaluation of the spatial policy. [pending: fill from `runs/train_spatial.log`]

tier	solved	mean coverage	mean k (solved)	k/k_{ref} (solved)
easy	[pending]	[pending]	[pending]	[pending]
mid	[pending]	[pending]	[pending]	[pending]
hard	[pending]	[pending]	[pending]	[pending]

4 MDP formulation and reward

Figure 7 summarises one step of the decision process. The rest of this section states each part precisely.

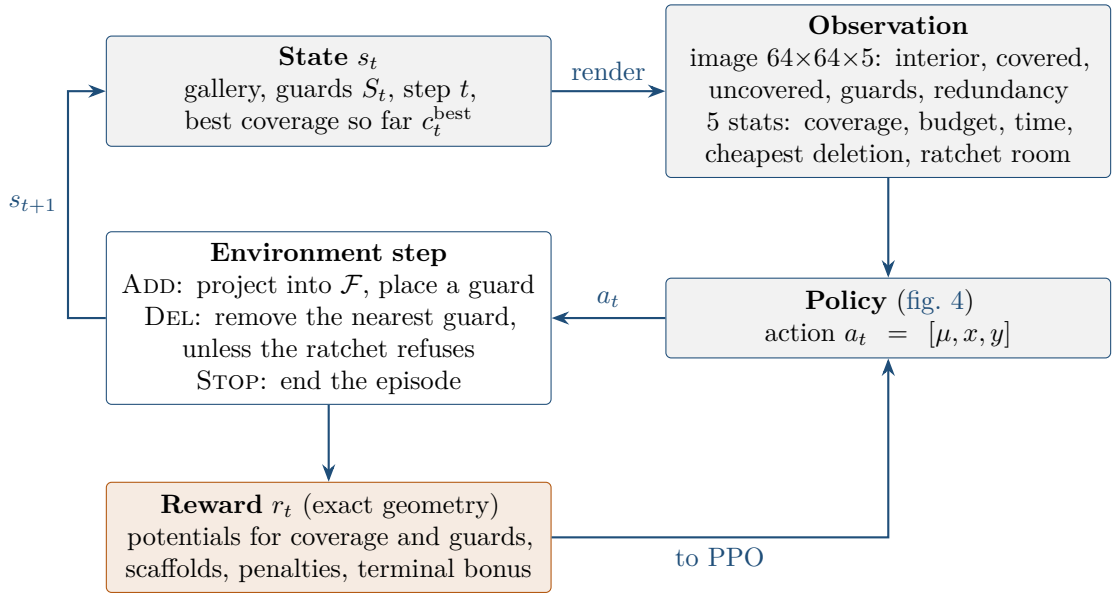


Figure 7: One step of the MDP. The state is rendered into a 5-channel image and five scalars; the policy outputs a mode and a point; the environment applies it with exact geometry and returns the next state and a reward. Transitions are deterministic; the only randomness is the gallery drawn at reset and the policy's own sampling.

4.1 Instances

Each episode draws a gallery uniformly from the training set and applies a random symmetry of the square (one of 8 rotations and reflections about the gallery centre), which also transforms the stored greedy guards. Augmentation is switched off for evaluation. The generator (`polygons.py`) produces three tiers (table 4).

4.2 State, actions, termination

The state is $s_t = (G, S_t, t, c_t^{\text{best}})$: the gallery, the multiset of placed guards, the step count, and the best coverage reached so far in the episode. The action $[\mu, x, y]$ is decoded to the point $q = (x_{\min} + x\,w,\ y_{\min} + y\,h)$.

Table 4: Difficulty tiers. 200 training and 30 test galleries per tier.

tier	outer vertices	holes	vertices per hole
easy	8–12	1–3	4–6
mid	10–16	2–5	4–8
hard	16–26	5–9	5–10

Add. If $q \notin \operatorname{int} \mathcal{F}$ (outside P or inside a hole), it is projected to the nearest boundary point and then moved towards a fixed interior reference point by the first $t \in \{0.01, 0.05, 0.10, 0.25, 0.50\}$ that lands strictly inside. A guard is *always* placed: rejecting the action instead would leave the state unchanged, and a deterministic policy that emits the same invalid point would repeat it forever. The projection is penalised (section 4.4) so the policy still learns the shape.

Del. Removes the guard nearest to q. Addressing a guard by a point rather than an index keeps the action fixed-size and independent of guard order, and reuses the same point head as ADD. Deletions are subject to a *coverage ratchet*: a deletion is refused (state unchanged, penalised) if it would bring coverage below $c_t^{\text{best}} - 0.01$. Undoing real progress is then impossible rather than merely costly.

Stop. Ends the episode with the current guard set.

Termination and truncation. The episode *terminates* on STOP; when an ADD reaches the budget of 16 guards; and, in stage 1 only, as soon as $f \geq c^{\dagger}$ (in stage 2 it continues so that pruning can happen). It is *truncated* after 40 steps. Truncation is a safety net, not part of the problem: no terminal reward is paid, and PPO bootstraps from the value function instead, which would double-count if we also paid out.

Why a budget of 16. Greedy needs at most 11 guards on any training gallery, so 16 leaves room for stage 1 to over-fill and stage 2 to prune. It is tight enough that guard placement still matters, and the observation reports the fraction of budget used. The reward sizes in section 4.6 are checked against this value and must be rechecked if it changes.

4.3 Observation

Table 5: Observation. Image channels are 0/255 masks except channel 4.

Image channels (64×64 over the bounding box)	
0 interior	pixels inside $\mathcal{F}$
1 covered	pixels seen by at least one guard
2 uncovered	interior pixels seen by no guard: the work left
3 guards	each guard as a 3×3 blob, so it survives strided convolutions
4 redundancy	each guard's blob shaded by $\min(1, \hat{\delta}_j^- / \sigma_p)$, where $\hat{\delta}_j^-$ is the raster estimate of its exclusive area and $\sigma_p = 0.005$; dark means nearly free to delete
Scalars (all clipped to $[0, 1]$)	
coverage	exact $f(S_t)$
budget used	$k_t/16$
time used	$t/40$
cheapest deletion	$\min(1, \min_j \hat{\delta}_j^- / \sigma_p)$; 0 means a free deletion exists now
ratchet room	$f(S_t) - (c_t^{\text{best}} - 0.01)$: coverage a deletion may still cost

Table 5 lists the observation. Channel 2 tells the policy where the work is; channel 4 is what makes DEL learnable at all, since without it a load-bearing guard and a redundant one

look identical. The raster estimate $\hat{\delta}_j^-$ counts pixels seen by guard j and by no other guard, one vectorised operation per guard. The exact δ^-, a polygon union over the other guards, is computed only when a deletion actually happens, for the reward and the ratchet.

4.4 Reward

Let g be the placed (projected) point, $\iota = \mathbf{1}[q \notin \operatorname{int} \mathcal{F}]$, D the bounding-box diagonal and $j^\star$ the guard nearest q. The per-step reward is

$$r_t = \begin{cases} w_{\text{cov}}\,\delta^+ - w_{\text{par}} - w_{\text{proj}}\,\iota + r_t^{\text{scaf}}, & \mu = \textsc{Add}, \\ -w_{\text{cov}}\,\delta_{j^\star}^- + w_{\text{par}} + w_{\text{prune}} \exp\!\left(-\delta_{j^\star}^-/\sigma_p\right), & \mu = \textsc{Del}, \text{ accepted}, \\ -w_{\text{void}}, & \mu = \textsc{Del}, \text{ refused or no guards}, \\ -w_{\text{stop}}\mathbf{1}\left[f(S_t) < c^\dagger\right], & \mu = \textsc{Stop}, \end{cases}$$

plus, on termination only,

$$r^{\text{term}} = \begin{cases} w_{\text{goal}} + w_{\text{pars}} \max(k_{\text{ref}} - k_T,\, 0), & f(S_T) \geq c^\dagger, \\ -w_{\text{fail}}, & \text{otherwise.} \end{cases}$$

where the scaffold reward of a placement is

$$r_t^{\text{scaf}} = w_{\text{front}}\mathbf{1}[g \notin \mathcal{C}(S_t)] + w_{\text{sweep}}\,\kappa_t(g) - w_{\text{sep}}\mathbf{1}[\min_j \|g - g_j\| < 0.15\,D]\,.$$

The sweep kernel favours the wall early and the interior later: $\kappa_t(g) = \exp(-(d_\partial(g)/D - m_t)^2/(2 \cdot 0.05^2))$ with $m_t = 0.02 + 0.23\,k_t/16$. Table 6 gives every weight; fig. 3 shows how they move over training.

Table 6: Reward terms and their weights (`rewards.py`). "Potential" marks the terms that are differences of the potential in section 2.6.

group	weight	role	SATURATE	PRUNE
coverage	w_{cov}	area gained / lost (potential)	1.0	1.0
	w_{goal}	terminal bonus for $f \geq c^\dagger$	4.0	4.0
	w_{fail}	terminal penalty otherwise	2.0	2.0
parsimony	w_{par}	cost per guard, refunded on deletion (potential)	0	0.15
	w_{prune}	bonus for a deletion that loses no area	0	0.20
	w_{pars}	terminal bonus per guard saved vs. k_{ref}	0	0.50
scaffolds	w_{front}	guard placed in uncovered space	0.04	0
	w_{sweep}	boundary-first, then inward	0.06	0
	w_{sep}	penalty for a guard within $0.15D$ of another	0.10	0
regularity	w_{proj}	aimed outside $\mathcal{F}$	0.25	0.25
	w_{void}	impossible or refused deletion	0	0.10
	w_{stop}	STOP before $c^\dagger$	0	2.0

Why each group exists. The coverage and parsimony potentials carry the objective (section 2.6). The terminal terms make crossing $c^\dagger$ the thing that matters, and w_{pars} rewards beating greedy, never punishing a match. The scaffolds make early coverage learning fast: aim at uncovered space, trace the boundary first, do not stack guards; being non-potential, they are removed before stage 1 ends. The regularity terms keep actions meaningful: aim inside the gallery, do not request impossible deletions, do not stop short.

4.5 Schedule

- **Stage 1.** The scaffolds hold full weight for the first 70% of the stage and then fall linearly to 0 at its end (fig. 3).
- **Stage 2.** The weights blend linearly from the stage-1 column (scaffolds at zero) to the stage-2 column over the first 5% of the stage, so the value function is not hit by a step change in its targets. 40% of stage-2 episodes start *presaturated*: the stored greedy cover plus 4 guards at random interior points. Those extras are redundant by construction, so free deletions exist from the first step and DEL gets a dense learning signal.
- A callback pushes the current weights to every environment every 4096 steps; the environment has no notion of stage beyond its weights and the `allow_delete` flag.

4.6 Weight calibration

The weights are sized so that no shortcut beats solving the gallery.

- **No scaffold farming (stage 1).** $w_{\text{front}} + w_{\text{sweep}} \leq 0.10$ per placement, so a full budget of 16 earns at most 1.6 in scaffolds, well below the 4 for reaching $c^\dagger$ plus the 2 for avoiding failure.
- **Parsimony never makes failure attractive (stage 2).** A full budget costs $16 \times 0.15 = 2.4$, less than the $w_{\text{goal}} + w_{\text{fail}} = 6$ swing between solving and failing.
- **Pruning is worth doing.** A free deletion returns $w_{\text{par}} + w_{\text{prune}} = 0.35$, large relative to per-step noise, and the bonus decays as $\exp(-\delta^-/0.005)$, by a factor e for every half percent of the gallery lost.
- **Stopping early is costly.** STOP below $c^\dagger$ costs $w_{\text{stop}} = 2$ plus the $w_{\text{fail}} = 2$ terminal penalty.

A known weakness: an add–delete cycle. The prune bonus is not potential-based. In stage 2, adding a guard in already covered space earns $-w_{\text{par}} = -0.15$, and immediately deleting it earns $w_{\text{par}} + w_{\text{prune}} = 0.35$: a net $+0.20$ per two steps, which over a 40-step episode can approach the goal bonus. The bonus is bounded per deletion but not per episode. Candidate fixes are to pay it only for guards that were present at the start of the episode or before the last ADD, to cap it per episode, or to anneal it to zero late in stage 2 as design rule 6 requires of non-potential terms. Stage-2 rollouts should be checked for this behaviour.

5 Optional stage 3: relocation

Stage 2 can only remove guards that are already redundant. It cannot move a guard so that a *neighbour* becomes redundant, which is where the remaining gap to the optimum lies, and which greedy, never revisiting a choice, cannot do either. A third stage would add a MOVE mode: select the guard nearest to the chosen point and displace it by a bounded continuous vector. Delete-then-add cannot substitute for it, because the ratchet refuses to delete any guard that alone covers more than 1% of the gallery.

What it would need:

- **A displacement head.** The spatial policy accommodates this naturally: a second pair of Beta maps per cell gives a bounded displacement for the guard selected in that cell, and the log-probability factorises as in section 3.3.
- **Reward.** The coverage potential already values a move correctly as $w_{\text{cov}}(f(S') - f(S))$, and the guard count does not change, so no new reward term is needed.
- **Ratchet interaction.** A move that lowers coverage below $c^{\text{best}} - 0.01$ should be refused like a deletion, or the slack widened for moves, since a good relocation can pass through

a temporary loss.

- **Constraint-based regularity.** The curriculum design proposes moving the regularity terms into a per-step constraint solved with N-P3O or CPO [2, 11], with threshold $h = 1.5\times$ the greedy solver's mean per-step regularity cost. A fixed penalty is negligible next to the first guard's gain but dominant next to the last sliver's, which suppresses exactly the delicate end-game moves stage 3 is for.
- **Hard-instance sampling.** Reweight the tiers towards hard galleries and sample instances in proportion to their recent failure rate.
- **A stronger reference.** Claiming a gap to the optimum rather than to greedy needs an exact or near-exact solver [5, 15] on a sample of galleries.

The prerequisite is that stage 2 prunes reliably. Relocation exists to remove guards that pruning cannot; if pruning does not yet work, adding MOVE doubles the continuous action dimension without addressing the actual bottleneck.

References

[1] Mikkel Abrahamsen, Anna Adamaszek, and Tillmann Miltzow. The art gallery problem is $\exists\mathbb{R}$-complete. In *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing (STOC)*, pages 65–73, 2018.

[2] Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 22–31, 2017.

[3] Po-Wei Chou, Daniel Maturana, and Sebastian Scherer. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 834–843, 2017.

[4] Václav Chvátal. A combinatorial theorem in plane geometry. *Journal of Combinatorial Theory, Series B*, 18(1):39–41, 1975.

[5] Marcelo C. Couto, Pedro J. de Rezende, and Cid C. de Souza. An exact algorithm for minimizing vertex guards on art galleries. *International Transactions in Operational Research*, 18(4):425–448, 2011.

[6] Stephan Eidenbenz, Christoph Stamm, and Peter Widmayer. Inapproximability results for guarding polygons and terrains. *Algorithmica*, 31(1):79–113, 2001.

[7] Uriel Feige. A threshold of $\ln n$ for approximating set cover. *Journal of the ACM*, 45(4): 634–652, 1998.

[8] Steve Fisk. A short proof of chvátal's watchman theorem. *Journal of Combinatorial Theory, Series B*, 24(3):374, 1978.

[9] Shengyi Huang and Santiago Ontañón. A closer look at invalid action masking in policy gradient algorithms. In *Proceedings of the International FLAIRS Conference*, volume 35, 2022.

[10] D. T. Lee and Arthur K. Lin. Computational complexity of art gallery problems. *IEEE Transactions on Information Theory*, 32(2):276–282, 1986.

[11] Joonho Lee, Lukas Schroth, Victor Klemm, Marko Bjelonic, Alexander Reske, and Marco Hutter. Evaluation of constrained reinforcement learning algorithms for legged locomotion. *arXiv preprint arXiv:2309.15430*, 2023. URL https://arxiv.org/abs/2309.15430.

[12] Rosanne Liu, Joel Lehman, Piero Molino, Felipe Petroski Such, Eric Frank, Alex Sergeev, and Jason Yosinski. An intriguing failing of convolutional neural networks and the CoordConv solution. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.

[13] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. An analysis of approximations for maximizing submodular set functions—i. *Mathematical Programming*, 14(1): 265–294, 1978.

[14] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pages 278–287, 1999.

[15] Davi C. Tozoni, Pedro J. de Rezende, and Cid C. de Souza. The quest for optimal solutions of the art gallery problem: A practical iterative algorithm. *ACM Journal of Experimental Algorithmics*, 21:1–23, 2016.

[16] Zhuoheng Wang, Jinyin Zhou, and Qi Wu. Dribble master: Learning agile humanoid dribbling through legged locomotion. *arXiv preprint arXiv:2505.12679*, 2025. URL https://arxiv.org/abs/2505.12679.

[17] Jimmy Wu, Xingyuan Sun, Andy Zeng, Shuran Song, Johnny Lee, Szymon Rusinkiewicz, and Thomas Funkhouser. Spatial action maps for mobile manipulation. In *Robotics: Science and Systems (RSS)*, 2020.